

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 5 : Session – 7 : SLO – 2 | Implementation of MST — Kruskal's Algorithm

■ I. Pseudocode Fill-in-the-Blanks

1. Sort all edges in ASCENDING (non-decreasing) order of weight.
2. Initialize parent array for all vertices (each vertex is its own parent).
3. Initialize MST as an empty set.
4. For each edge (u,v) in sorted order:
 - a. If $\text{find}(u) \neq \text{find}(v)$: i. Add (u,v) to MST. ii. Perform UNION operation.
5. Stop when MST has n-1 edges.

■ II. Complete the C Code Snippet

```
• int findParent(int parent[], int i) {  
• if (parent[i] != i)  
• parent[i] = findParent(parent, parent[i]); // Path compression  
• return parent[i];  
• }
```

■ III. Predict the Output

- Edges sorted: A-B(2), B-C(3), A-C(4).
- Accept A-B(2); accept B-C(3); A-C would form a cycle -> skip.
- Total MST weight = 2 + 3 = 5.

■ IV. Identify Cycle Formation

- No — A-C (3) must NOT be included. A and C are already connected through A-B-C,
- so adding A-C would create the cycle A-B-C-A. Kruskal's accepts an edge only if $\text{find}(u) \neq \text{find}(v)$.

■ V. Pseudocode Error Spot

- Wrong: the snippet adds the edge when $\text{find}(u) == \text{find}(v)$ — that ADDS a cycle.
- Fix: if ($\text{find}(u) \neq \text{find}(v)$) { Add edge (u,v) to MST; Union(u, v); } else { Continue; }

■ VI. Creative Reasoning

- Kruskal's is preferred for sparse graphs because its cost is dominated by sorting the edges, $O(E \log E)$,
- and with few edges ($E \ll V^2$) this is far cheaper than Prim's $O(V^2)$ array version; the Union-Find
- structure also handles the disconnected forest gracefully without scanning whole rows of a matrix.